BLEND: A Unified Data Discovery System

1st Mahdi Esmailoghli
TU Berlin & BIFOLD
Berlin, Germany
esmailoghli@tu-berlin.de

2nd Christoph Schnell
Leibniz Universität Hannover
Hannover, Germany
schnell@dbs.uni-hannover.de

3rd Renée J. Miller
University of Waterloo
Waterloo, Canada
rjmiller@uwaterloo.ca

4th Ziawasch Abedjan
TU Berlin & BIFOLD
Berlin, Germany
abedjan@tu-berlin.de

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00061

***Abstract*—Most research on data discovery has so far focused on improving individual discovery operators such as join, correlation, or union discovery. However, in practice, a combination of these techniques and their corresponding indexes may be necessary to support arbitrary discovery tasks. We propose BLEND, a comprehensive data discovery system that supports existing operators and enables their flexible pipelining. BLEND is based on a set of lower-level operators that serve as fundamental building blocks for more complex and sophisticated user tasks. To reduce the execution runtime of discovery pipelines, we propose a unified index structure and a rule- and cost-based optimizer that rewrites SQL statements into low-level operators when possible. We show the superior flexibility and efficiency of our system compared to ad-hoc discovery pipelines and stand-alone solutions.**

I. Introduction

The potential for large structured data lakes, such as governmental data lakes [19], [61], enterprise-level corpora [10], [15], and public-access lakes [22], [14], [33], has spurred various avenues of research, all aimed at empowering data scientists with the ability to extract further insights for their diverse tasks, such as machine learning (ML) [23], exploratory data analysis [41], and data cleaning [40]. To support these tasks, various data discovery operatorion have been studied, such as **join discovery** [24], [69], [63], [62], [59], [27], [70], [20], [16], [56], [50], **union search** [45], [11], [36], [13], [25], [34], and **correlation discovery** [23], [49], [9], [17].

Existing research has focused on each operator individually without considering possible opportunities for combining, pipelining, and optimizing them. Yet, fulfilling a real-world data discovery task often requires a combined application of these operators. For instance, in the case of data enrichment for ML, the user needs to use a combination of join, union, and correlation discovery [9], [45]. To achieve this, we need a unified data discovery system.

Requirements for a unified discovery system. A trivial approach to building such a unified system is to federate all indexes and algorithms via a common interface. Even after overcoming the inherent software and interface heterogeneity among prototypes, the execution of complex data discovery tasks that combines these operators across systems is inefficient and inflexible in terms of both storage and maintenance. This stems from several key challenges:

Lack of Flexibility. Introducing new criteria or discovery operators is challenging since new indexes and algorithms must be integrated into an already complex system.

Storage and Maintenance Overhead. Multiple independently developed index structures must be generated and stored. The aggregated system will comprise an incoherent set of potentially redundant structures needing maintenance.

Lack of Optimization. The federation of independent operators under the same abstraction does not lend itself to optimization. To showcase the operational complexity of a rather simple discovery task, consider following example:

Example 1. Fig. 1 illustrates a user table S along with three tables $T1$, $T2$, and $T3$ from a data lake. S contains information about departments in a company and the name of their corresponding leaders, i.e.,"Head". The user wants to fill in missing values of S but already knows that the head of the IT department, "Tom Riddle", has left the company. Therefore, any table that contains the projection ("IT","Tom Riddle") is outdated. Thus, the user defines the data discovery for missing values as follows: *Find top-*1 *table that contains ("HR" and "Firenze" in a row and "HR", "Marketing", "Finance", "IT", ... in a column but does not contain rows with "IT" and "Tom Riddle").*

To solve this task with existing discovery operators, one could use single-column join discovery, such as Josie [69], to look up overlapping column values, and multi-column join discovery, such as MATE [24], to find overlapping column combination values. With MATE, one could find tables containing the projection ("HR", "Firenze") as result set rs_1. Also, MATE can be used to identify tables containing the tuple ("IT", "Tom Riddle") as result set rs_2. Josie can be employed to find tables that join only on the department values "HR", "Marketing", "Finance", ... as result set rs_3. Finally, the equation $((rs_1 \cap rs_3) - rs_2)$ is the answer to the user's query. According to Fig. 1, $rs_1 = \{T2, T3\}$, $rs_2 = \{T2\}$, and $rs_3 = \{T1, T2, T3\}$. Thus, the answer to the discovery query is $\{T3\}$, which contains amendable values for the "Head" of the department column in S. •

The example illustrates that running independent discovery redundant and inefficient. For each look-up, combinations of the three tables $T1$, $T2$, and $T3$ may be processed. However, knowing that $T1$ and $T2$ are not desired tables, we could exclude them from follow-up operators: first, by checking all three tables to find those containing ("HR" and "Firenze"); then, by focusing only on $T3$ and $T2$ and dropping the outdated $T2$; and finally, by verifying $T3$ for overlapping department names. Intuitively, we could obtain the same results by resorting to application code. We aim to

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00061

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.



Fig. 1: Discovering up-to-date tables to fill in the NULLs.

achieve the same flexibility within a discovery system with a unified language to optimize custom discovery processes. Such abstraction becomes necessary in case of complex queries with a variety of operators and result sets with millions of tables.

Contributions We introduce BLEND, a unified system that enables users to create complex discovery plans. With no need for compiling, installing, or adapting additional discovery systems, BLEND streamlines the discovery process. Given a data discovery plan, a data lake, and a parameter k, BLEND retrieves the top-k tables from the lake that are most relevant to the defined plan. Our key contributions are as follows:

1) **Introduction of an abstraction layer:** We design a novel abstraction that unifies the building blocks of different discovery tasks. We propose a set of atomic search operators, which users can declaratively compose to efficiently discover relevant tables. These building-blocks are implemented using primitive database functions, allowing the majority of computations to run within the database.
2) **Optimized execution of tasks:** We develop a two-phase plan optimizer that combines rule- and cost-based query optimization techniques. It re-orders the execution of operators based on predefined rules, ensuring efficient execution while preserving the intended logic of the discovery task. Furthermore, BLEND optimizes individual operators by rewriting corresponding queries using intermediate results.
3) **Integrated index:** We studied fifty-three index structures from thirty data discovery papers and propose a unified index that efficiently supports the execution of the introduced operators and their compositions. The index is designed as a single database table, enabling the system to additionally benefit from the database-level query optimizations.
4) **Simplification of complex tasks using SQL operations:** Our proposed index structure allows for the translation of BLEND's operators into SQL queries. Even complex tasks, such as correlation discovery, are reduced to SQL-based operations, eliminating the need for custom algorithms or external implementations. By integrating multiple operators into declarative pipelines, BLEND minimizes user effort and simplifies the creation of discovery tasks.
5) **User study:** We conducted a user study with data experts to understand their discovery preferences and evaluate the effectiveness of BLEND in addressing these needs.

II. Problem Statement

Given the requirements formulated in the introduction, we formalize the following problem statement. Let $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$ be a data lake, where each T_i is a table. Given a query table S with insufficient data and a discovery plan P by the user, the goal of BLEND is to enable the formulation of the plan and its optimized compilation for execution. The result of the execution is the subset $\mathcal{T}' \subseteq \mathcal{D}$ that enriches S. With this functionality in mind, we strive to solve four technical problems (**Pr.1-4**):.

First, the language for defining a plan should support the formulation of the most prominent discovery operations from the literature (**Pr.1**):

- **Keyword search**: discovered tables must overlap with a given set of keywords.
- **Join discovery**: discovered tables must be joinable with S on one or more columns.
- **Union discovery**: rows in discovered tables must be unionable with rows in S.
- **Correlation discovery**: a column in each discovered table must correlate with the user-defined numerical target in S.

Second, the system should also support any set-based aggregation of these operation's results (**Pr.2**).

Third, we require an index that serves all of the possible discovery operations but is leaner than the ensemble of the required individual indexes. More formally, assuming that an independent system exists for each of the aforementioned discovery operations M_{op_i}, where $1 < i < n$, the storage required for our index to execute any combination of operators in OP must be smaller than $\sum_{i=1}^{n} \text{Storage}(M_{op_i})$, where $Storage(M_{op_i})$ is the storage size required for the state-of-the-art index in M_{op_i} (**Pr.3**).

Finally, the system should satisfy two runtime constraints. The implementation of each (M_{op_i}) in the unified system should yield a comparable runtime while any aggregation and combination of more than two M_{op_i} should yield superior runtime compared to their execution as concatenated black box discovery system. Formally, assuming a discovery plan P is an arbitrary combination of operators in OP, i.e, $P \in OP^*$, the execution runtime of P in BLEND should satisfy

$$\text{Time}(P_{\text{BLEND}}) \leq \sum_{j|op_j \in P} \text{Time}(M_{op_j}).$$

Satisfying the previous constraint should not be a burden on how the user formulates the plan. BLEND must identify a good (near optimal) ordering of the operations based on the expected runtime and intermediate result size (**Pr.4**)

III. System Overview

Fig. 2 depicts an overview of BLEND's architecture depicting its online and offline phases. The online phase involves four stages: task definition, plan formulation, plan optimization, and execution.

First, the user defines their data discovery task using BLEND's API (Fig. 2a). Here, the plan *find_dep_heads* is

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.

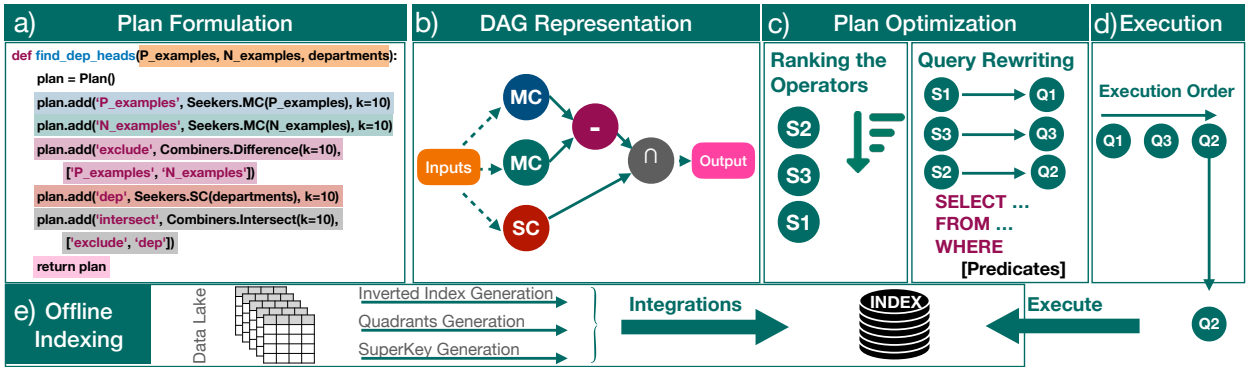


Fig. 2: BLEND's architecture.

written for the use case of Fig. 1. It finds tables containing a set of positive examples and the given department names but not negative examples. The main primitives in BLEND's API are *Plan*, *Seekers*, and *Combiners*. The latter two, *Seekers* and *Combiners*, are the lower-level operators in BLEND, which are implemented in SQL. A discovery task is created by instantiating a *Plan* object, to which one can add *Seekers* and *Combiners*. Seekers are added by specifying an identifier string and parameters, such as input lists and top-k. BLEND transforms the plan into a directed acyclic graph of operators, where edges represent the data flow (Fig. 2b). Once the discovery plan is written and the DAG is generated, BLEND leverages a combination of rule-based and cost-based optimization to re-rank and rewrite operators for faster execution (Fig. 2c). The low-level operators either resemble or contain SQL queries on BLEND's index which is organized as a fact table. Based on the cost model, BLEND rewrites the SQL statements by injecting predicate placeholders enabling the system to take advantage of cardinality estimates of the intermediate results. Once the queries are generated, they are sent to the execution engine to run in the database (Fig. 2d). In the offline phase, BLEND builds the necessary index and data structures (Fig. 2e). The index comprises and unifies three structures: inverted index [5], *super keys* [24], and quadrants [49].

IV. OPERATORS

BLEND's operators are designed to be composable, enabling complex plans, and simple for efficient execution. BLEND leverages two sets of operators: seekers and combiners.

A. Seeker operators

A seeker receives a set of columns, Q, as the input and returns the top-k most relevant tables. k is a user-defined parameter. We propose four seekers: *single-column*, *keyword*, *multi-column*, and *correlation* seekers.

1) Single-Column (SC) Seeker: The SC seeker receives an input column Q, typically from an input (query) table and returns a list of k tables. These tables contain a column that overlaps the most with Q (sorted by number of overlaps).

2) Keyword (KW) Seeker: The KW seeker returns the top-k overlapping tables based on a given set of keywords. The main difference between the KW and SC seekers is that the KW seeker measures the overlap throughout entire candidate tables rather than single columns. Returned tables are sorted based on the number of overlaps.

3) Multi-Column (MC) Seeker: The MC seeker receives Q that has multiple columns and discovers the top-k tables in the data lake that the most with the tuples in Q. The returned tables are sorted based on the overlap.

4) Correlation (C) Seeker: This seeker receives two columns Q, $Q = \{Q_j, R\}$, where Q_j is a join key and R is a numerical target column. The correlation seeker returns the top-k tables that join on Q_j and contain a column that correlates the most with R, sorted by the correlation coefficient.

B. Combiner operators

Combiners enable the discovery task specific composition of seeker results. Combiners receive a set of table collections, where each collection is the result of a seeker or another combiner. A combiner merges the table collections according to its defined set operation and returns a new set of tables. We implemented the most common set operators in BLEND: *Intersection* ($\cap$), *Union* ($\cup$), *Difference*($\setminus$), and *Counter*. Note that the user can introduce new combiners to the system. The **Intersection** and **Union** combiner receive the output of two or more seekers and returns the intersection/union of the tables, respectively. The **Difference** combiner receives only two sets of tables and returns the tables that exist only in the first table set. The **Counter** combiner receives multiple sets of tables, counts the occurrence of the table identifiers, and returns a list of tables sorted in descending order based on their frequencies.

C. Discovery Language Grammar

With the given seekers and combiners, the grammar of the supported discovery plans in BLEND can be defined as follows:

```
expression ::= seeker(Q) |
combiner(expression(,expression)+)
seeker ::= KW | SC | MC | C
combiner ::= ∩ | ∪ | \ | Counter
Q ::= keyword | table
```

Any `expression` based on the grammar above can be subject to optimization of BLEND. Further, it is possible to

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:39:53 UTC from IEEE Xplore. Restrictions apply.



CHORUS: Foundation Models for Unified Data Discovery and Exploration

Moe Kayali
kayali@cs.washington.edu
University of Washington

Anton Lykov
alykov@cs.washington.edu
University of Washington

Ilias Fountalis
ilias.fountalis@relational.ai
RelationalAI

Nikolaos Vasiloglou
nik.vasiloglou@relational.ai
RelationalAI

Dan Olteanu
olteanu@ifi.uzh.ch
University of Zurich

Dan Suciu
suciu@cs.washington.edu
University of Washington

ABSTRACT

We apply foundation models to data discovery and exploration tasks. Foundation models are large language models (LLMs) that show promising performance on a range of diverse tasks unrelated to their training. We show that these models are highly applicable to the data discovery and data exploration domain. When carefully used, they have superior capability on three representative tasks: table-class detection, column-type annotation and join-column prediction. On all three tasks, we show that a foundation-model-based approach outperforms the task-specific models and so the state of the art. Further, our approach often surpasses human-expert task performance. We investigate the fundamental characteristics of this approach including generalizability to several foundation models and the impact of non-determinism on the outputs. All in all, this suggests a future direction in which disparate data management tasks can be unified under foundation models.

PVLDB Reference Format:
Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. CHORUS: Foundation Models for Unified Data Discovery and Exploration. PVLDB, 17(8): 2104 - 2114, 2024.
doi:10.14778/3659437.3659461

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at http://github.com/mkyl/CHORUS.

1 INTRODUCTION

Data discovery and exploration are major components of the workflow of analysts and data scientists. A survey conducted by the Anaconda data-science platform in 2021 found that analysts spend 40% of their working hours on data loading and cleaning [2]. Even with this colossal effort, 60-70% of data within an enterprise still goes unused for analytics [21], remaining as *dark data* [23, 63].

Recent developments in large language-models (LLMs) have unlocked human-level performance on diverse domain tasks. The discovery that these models can generalize to diverse domain-specific tasks that they have not been trained on [3, 26, 59, 60] has led to emergence of the term *foundation models* [5].

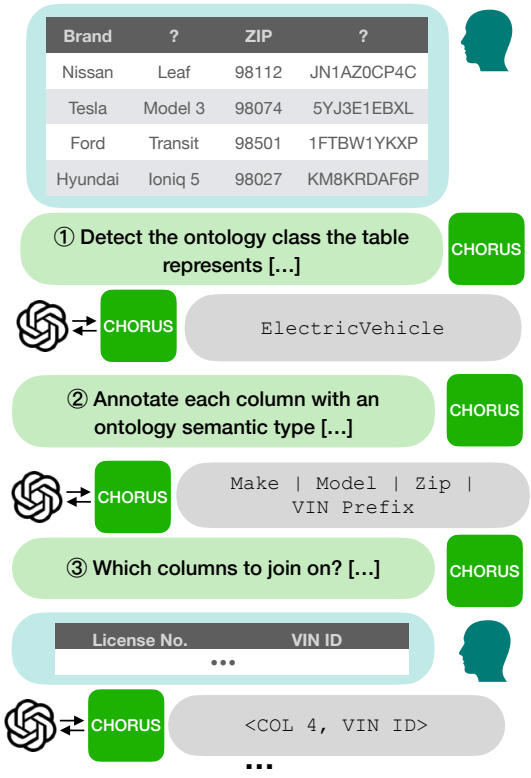


Figure 1: Data discovery tasks considered in this work. Given an ontology, such as DBPedia, ① we assign an overall type to the table and ② we annotate the columns with semantic types. Last, given another table, ③ we predict the join column. The user provides the data while CHORUS interacts with the foundation model. Data from [44], full prompts in Figure 3.

Despite their promise, serious risks have hampered the reception of foundation models. These include: spurious generation (including "hallucination") [24], factual recall limitations [39], bias [19], dataset contamination [14], logical shortcuts [50] and fallacies [38]. Naïve deployment can lead to unanticipated problems: it has already led to legal action [11] and recalls by major corporations [22]. These risks are now acknowledged by the creators of these models [6, 45, 54].

The goal of this paper is to demonstrate the utility of foundation models to the data discovery and exploration while mitigating

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 8 ISSN 2150-8097.
doi:10.14778/3659437.3659461

the aforementioned risks. We select three representative tasks to show the promise of foundation models: ① *table-class detection*, ② *column-type annotation* and ③ *join-column prediction*. An outline of our approach is shown in Figure 1. We call our approach CHORUS.

Contributions. We summarize our contributions:

- The first work to use foundation models for the data discovery tasks of table-class detection, column-type annotation and join-column prediction;
- Propose a novel system, CHORUS, whose flexible architecture enables the synthesis of multiple data discovery tasks and deploying risk mitigations;
- Design task-specific approaches that exploit zero- and few-shot strategies and allow information flow between tasks;
- Introduce the novel mitigation of *anchoring* to reduce foundation-model risks specific to this domain;
- Empirically validate CHORUS, comparing its performance with the state-of-the-art baselines across three individual tasks.

Discussion. Prior work has addressed these tasks individually. Landmark approaches like Sherlock [27] trained deep model architectures for a specific task, requiring 100K-1M labeled data points. More recent work such as DoDuo [52] and TaBERT [62] has focused on *representation learning*, learning embeddings for structured data by improving their performance on one or more downstream tasks.

Foundation models allow a substantially different approach: rather than the classical architecture where the outputs of the model are task-specific, the inputs and outputs of the model are natural language text. Training occurs not on tables or data management tasks specifically, but on general text. Performance on domain-specific tasks is solely by generalization.

This results in a high degree of flexibility. Novel tasks can be specified in natural text, without need for expensive data collection—task examples, metadata and constraints are all incorporated into the task easily. Another advantage of this approach is a **unified architecture**: tasks can utilize the overall context and previous outputs. For example, in Figure 1 the table class `ElectricVehicle` helps with deducing the outputs `Make`, `Model` in the next task.

Outline. Section 2 defines the three tasks investigated in this paper. Section 3 describes the architecture of CHORUS and key approaches. We evaluate the performance of CHORUS in Section 4's experiments. In that section, we also investigate the fundamental characteristics of this approach. We offer a discussion of those results in Section 5. This includes a discussion of promising future directions. Finally, we place this work within the literature in Section 6, discussing related works.

2 BACKGROUND

2.1 Tasks

We assume to be given a *data collection* consisting of a number of relational tables T_1,T_2,.... Each table T_i consists of a number of columns, or attributes, A_1,A_2,... and a number of rows, or tuples, r_1,r_2,... The name of a table T_i is, in general, non-informative, for example it may be simply a sequential ID. The columns may optionally have a name H_1,H_2,... or consist only of values.

In addition to the data collection, we are also given a reference ontology of table classes C_1,C_2,..., and a reference ontology of column types τ_1,τ_2,.... For example, the DBPedia.org types for the table classes include https://dbpedia.org/ontology/Actor and https://dbpedia.org/ontology/Continent and column types include https://dbpedia.org/ontology/areaTotal and https://dbpedia.org/ontology/birthDate.

We consider three tasks of interest on the data collection:

Definition 2.1 (① Table-class detection)**.** For each table T_i, determine its appropriate class C_j, such that every row r_1,r_2,... represents an instance of the C_j type. We adopt this definition from [33].

For example, table-class detection on the table given in Figure 1 could output `ElectricVehicle`, since each row of that table is an instance of that class. Alternatively stated, the table is about `ElectricVehicles`.

Definition 2.2 (② Column-type annotation)**.** For each table T_i, find a mapping from its attributes (columns) A_1,A_2,... to the reference column types τ_1,τ_2,..., such that each value in A_i is an instance of the τ_i type. See [1, 13].

For example, column-type annotation on the first column in Figure 1 could output `Manufacturer`, since the values are the respective manufacturers of each `ElectricVehicle`.

Definition 2.3 (③ Join-column prediction)**.** Assume an *execution log* L, a history of user actions including table joins and their join conditions, which maps many $(T_i,T_j) \rightarrow (A_k,A_l)$ where $A_k \in T_i, A_l \in T_j$. Given two tables T and T', with columns A_1,... and A'_1,... respectively, the *join-column prediction* task is to suggest a pair (A_k,A'_l) of columns such that the equality condition $A_k = A'_l$, which can be used to join the the tables, matches with the choice in the execution log L. For more discussion, see [61].

For example, given the table in Figure 1 and another table `car_registration(name, vehicle_id_number)`, join-column prediction could output `(VIN_prefix, vehicle_id_number)`. The correctness of the prediction depends on the ground truth of which columns the user did in-fact join on.

Ontologies Foundation models contain knowledge of ontologies such as DPBedia.org, Freebase and Wikidata. We focus on universal ontologies, that is, ontologies that aim to represent all entities in general. This is in-line with findings that foundation models encode highly technical knowledge, such as clinical reasoning [51] or electrical engineering principles [53].

3 APPROACH

We outline the structure of CHORUS in this section. First, we explore the core idea of ingesting relational data with foundation models and performing data exploration tasks in Subsection 3.1. Next, we describe the necessary post-processing and mitigations we develop in Subsection 3.2.

Figure 2 shows the architecture of the system. CHORUS has a unified architecture which runs multiple tasks in the same context, allowing for information flow. Each task is run sequentially, with the output of one task fed as context into future tasks.

For each task instance, CHORUS generates a prompt by concatenating six inputs: context, demonstration, data samples, metadata,

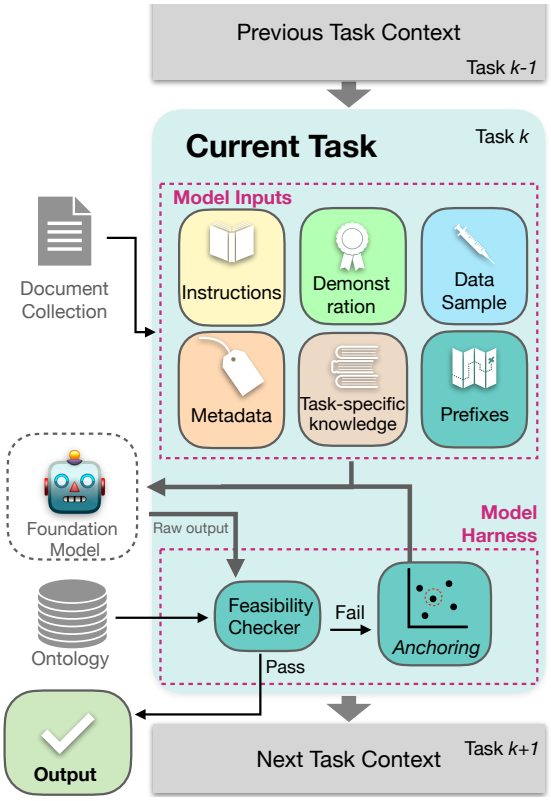


Figure 2: CHORUS system architecture.

task-specific knowledge, and prefixes. They form the "Model Inputs" box in Figure 2 and are color-coded so that they match the colored prompt components in Figure 3. This natural language input is then fed to the foundation model. The output is then subject to post-processing: checks of parsability and feasibility are conducted. If these pass, the output is extracted. Otherwise, we activate a mitigation process, called *anchoring*, in order to repair the error and prevent its propagation.

3.1 Model Inputs

We discuss what inputs are provided to the foundation model and how they are pre-processed and synthesized. We discuss the six components of the Model Inputs module in Figure 2, individually. These correspond to the six color-coded prompt components in Figure 3. Once generated, all the above inputs are concatenated to into a single prompt provided to the model.

Instructions. A description of the specific task (table-class detection, column-type annotation or join-column prediction) is provided to the foundation model in natural language. These are shown in yellow in Figure 3. For example, we translate the formal Definition 2.1 of the first task, table-class detection, into the English sentence "For the following CSV sample, select one DBpedia.org ontology that represents the dataset." For the third task, join-column prediction, we utilize a code-completion approach. We frame the task as code-completing a Pandas fragment that performs a join, with the code to complete shown in Figure 3c. We choose Pandas because it is a very popular framework, with more than millions of example lines of code

Legend: Instruction, Demonstration, Data sample, Metadata, Task-specific knowledge, Prefixes.

```
For the following CSV sample, select one DBpedia.org ontology that
represents the dataset from the following list:
AcademicJournal, AdministrativeRegion, Airline, Airport, [. . . ],
University, VideoGame, Work, Wrestler.
For example, for a dataset about hospitals, return
`https://dbpedia.org/ontology/Hospital`. Begin your answer with
'https://dbpedia.org/ontology'.
```
```
Brand, ␣, ZIP, ␣,
Nissan, Leaf, 98112, JN1AZ0CP4C,
Tesla, Model 3, 98074, 5YJ3E1EBXL,
[...]
```
```
```

(a) Table-class detection

```
Consider this example. Input:
```
```
Name, Famous Book, Rk, Year
Fyodor Dostoevsky, Crime and Punishment, 22.5, 1866
Mark Twain, Adventures of Huckleberry Finn, 53, 1884
Albert Camus, The Stranger, -23, 1942
```
```
Output:
`dbo:author, dbo:title, Unknown, dbo:releaseDate`.
For the following CSV sample, suggest a DBPedia.org Property for each
column from the `dbo:`namespace.
```[...]```
```

**(b) Column-type annotation**

```
Given two Pandas Dataframes, suggest what `pd.merge`
parameters to use to join the dataframes.
df1 =
```[...]```
df2 =
```[...]```
Complete the correct Pandas merge command. `pd.merge(df1, df2, left_on=
```

**(c) Join-column prediction**

**Figure 3: Prompts used in this paper, materialized with examples. Most prompt elements are fixed—only the data sample and metadata change for each instance.**

on the web. This is the *zero-shot prompt* setting: the model can be provided with instructions for a novel task and performs them directly.

*Demonstration.* For the first two tasks, we use the foundation models with task examples as an additional input: this is called the *few-shot prompt* setting. The model is given a few demonstrations of task completion, including inputs and outputs. This is shown in Figure 3 as green text.

*Data sample.* By serializing the input tables, we can input them into foundation models. For example, consider the example table from Figure 1 in the introduction. Serializing the table allows the foundation model to ingest the data. We use the comma-separated values (CSV) format, shown in blue in Figure 3.

Because the models have a limited context window size—typically in the few thousands of tokens—tables cannot always be ingested as a whole. Instead, we always serialize a sample of the rows. We find
```

LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes

Chengliang Chai*
ccl@bit.edu.cn
Beijing Institute of Technology

Yuhao Deng*
dyh18@bit.edu.cn
Beijing Institute of Technology

Yutong Zhan
zyt@bit.edu.cn
Beijing Institute of Technology

Ziqi Cao
caoziqi@bit.edu.cn
Beijing Institute of Technology

Yuanfang Zhang
zyuanfang@bit.edu.cn
Beijing Institute of Technology

Lei Cao
lcao@csail.mit.edu
University of Arizona/MIT

Yuping Wang
wyp_cs@bit.edu.cn
Beijing Institute of Technology

Zhiwei Zhang
zwzhang@bit.edu.cn
Beijing Institute of Technology

Ye Yuan
yuan-ye@bit.edu.cn
Beijing Institute of Technology

Guoren Wang
wanggr@bit.edu.cn
Beijing Institute of Technology

Nan Tang
nantang@hkust-gz.edu.cn
HKUST (GZ)

ABSTRACT

Searching tables from poorly maintained data lakes has long been recognized as a formidable challenge in the realm of data management. There are three pivotal tasks: keyword-based, joinable and unionable table search, which form the backbone of tasks that aim to make sense of diverse datasets, such as machine learning. In this demo, we propose LakeCompass, an end-to-end prototype system that maintains abundant tabular data, supports all above search tasks with high efficacy, and well serves downstream ML modeling. To be specific, LakeCompass manages numerous real tables over which diverse types of indexes are built to support efficient search based on different user requirements. Particularly, LakeCompass could automatically integrate these discovered tables to improve the downstream model performance in an iterative approach. Finally, we provide both Python APIs and Web interface to facilitate flexible user interaction.

PVLDB Reference Format:
Chengliang Chai, Yuhao Deng, Yutong Zhan, Ziqi Cao, Yuanfang Zhang, Lei Cao, Yuping Wang, Zhiwei Zhang, Ye Yuan, Guoren Wang, and Nan Tang. LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes . PVLDB, 17(12): 4381 - 4384, 2024.
doi:10.14778/3685800.3685880

1 INTRODUCTION

Many organizations have acknowledged the importance of providing open access to their tabular data for the public good [14]. However, the large amount of tables are often poorly maintained (*e.g.*, lacking of index or schema information). To make sense of this data, a key initial step for the data scientists is to search tables relevant to the task at their hand. To this end, we propose to build an end-to-end prototype system, LakeCompass that driven by the need of the downstream analytics tasks, offers efficient and effective table search over data lakes.

At a high level, LakeCompass features the following three key functionalities.

Index building. We build various types of indexes over tables in data lakes, such as inverted index, local sensitive hash (LSH) and the hierarchical navigable small world (HNSW), which taking both text similarities and semantic similarities into consideration, efficiently and effectively support table search.

Table search. We support three typical categories of table search, *i.e.*, keyword-based, joinable and unionable table search. The former one returns relevant tables based on the given keywords. The latter two take as input a query table, and discovering tables that can be integrated with the query table.

Data Selection for Model analysis. LakeCompass offers a data selection component that taking into consideration the specific need of the downstream applications, *i.e.*, the ML models, intelligently selects the tuples and features from the table search result in a more targeted manner.

Overall, LakeCompass has the following advantages.

[1. *High efficiency.*] Various types of indexes allow us to support efficient search over different queries. In terms of the model analysis module, we propose to use the online learning technique to improve the efficiency of iterative model evaluation.

[2. *High effectiveness.*] We maintain the original data associated with their semantic embeddings, together with various indexes, which allow users to search highly relevant tables customized to their needs. For the model analysis, we propose a multi-armed bandit (MAB) technique to judiciously select beneficial tuples/features among the returned relevant tables.

[3. *User-friendly interaction.*] We encapsulate the above functionalities through flexible Python APIs, where the user can upload

*Both authors contributed equally to this research.
Ye Yuan is the corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 17, No. 12 ISSN 2150-8097.
doi:10.14778/3685800.3685880

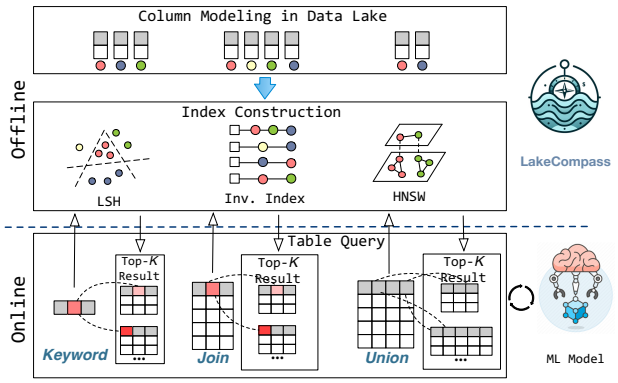


Figure 1: Overall Architecture of LakeCompass.

data, construct indexes, search tables and build ML models. We also provide a Web interface to serve table search and analysis in a more intuitive way.

Existing Works. There exist several prototypes of table search. Aurum [6] takes as input keywords and returns relevant tables based on their schema information. COCOA [5] focuses on searching joinable tables; Auctus [1] supports both joinable and unionable table search; Besides the above table search, Juneau [7] further supports feature extraction and data cleaning based on the returned tables. Similarly, DIALITE [12] can also search tables that can be integrated with the query table, and then conducts some data analysis like entity matching. Although the above prototypes supports table search in data lakes, they do not consider the semantic information of columns/rows of tables, which is rather significant in achieving high effectiveness. What's more, they do not take the machine learning task into consideration, which is the key application for table search in data lakes. Although some works [3, 9] have studied data augmentation for ML, they assume the relevant data is discovered beforehand by another system, hence not offering an end-to-end table search solution.

In summary, LakeCompass is an end-to-end system that is adequate to demonstrate the efficiency and effectiveness of table search for real-world applications.

2 SYSTEM OVERVIEW

At a high level, LakeCompass consists of the following modules: data lake embedding, index construction, online table query processing as well as the model analysis, as shown in Figure 1. The former two modules are offline, which respectively encode the column/schema in the data lake into a vector and index these vectors. Once an online query table arrives, we first encode its column(s) and then use the index to retrieve relevant tables with the highest relevance score from the data lake. If the query is a keyword, we just encode the query and retrieve relevant tables.

Embedding Data Lake. Generally speaking, column representations are significant in table search. The columns of the original tables in data lake are always represented as fixed-length vectors. The vectors are mostly hash codes [6, 17] (*e.g.*, generated by the minhash function) or embeddings [8, 10] (*e.g.*, generated by pre-trained language models) that are effective in finding columns with similar semantics or high overlaps, which are rather significant in searching joinable/unionable tables. However, some methods like Josie and InfoGather [15, 16] directly use the original cell values of each column to search highly overlapping columns rather than using vectors. Besides, we also maintain the metadata of data lake tables (*e.g.*, table name, caption and schema information, etc.) and index them, which are also important in table search, especially for keyword-based search.

Index Construction. Almost all table search methods rely on an index to search large table repositories. Building upon the representations discussed above, several typical ANN indexes like LSH [17] or HNSW [8] can be utilized to index all the embeddings. As another option, inverted index [11, 15, 16] accelerates the process of finding highly overlapping columns, where each cell value is linked to the columns that contain the value. In this case, these colorful circles represent Column IDs rather than vectors. All the types of indexes are supported in LakeCompass.

Online Table Search. For keyword-based search, the user issues the keywords, which are encoded as embeddings, and retrieve tables with similar embeddings on metadata or columns. For joinable search, LakeCompass supports users in issuing a query table through either user-uploaded files or the result of keyword-based search. Once a user issues a table T_J and a specific column c for search, LakeCompass first represents c as a vector. It then uses the index to quickly identify a of top-K columns (tables), denoted by $R(T_J)$, with high relevance scores. The unionable search follows the same procedure. The only difference is that given a table T_U, it has to search over each attribute in T_U and aggregate the results. Specifically, for each column $c_i \in T_U$, similar to the join search, we retrieve from the data lake a set of columns that have high relevance scores with c_i. Then, we compute the union of all retrieved tables. For each table T in the union set, we compute the relevance score using techniques like maximum bipartite graph matching, considering the column relevance between the T_U and T. Finally, a set of top-K tables, denoted by $R(T_U)$ with the highest relevance scores are returned. In LakeCompass, we have implemented various joinable [15–17] and unionable search [8, 11, 15] algorithms.

Iterative Data Selection for Model Analysis. When the query table serves as the train data, due to the data scarcity problem, we expect to augment more tuples/features through unionable/joinable table search. However, although all the returned tables $R(T_J)/R(T_U)$ can be integrated into the query table, integrating all of them may not improve the model performance. The reason is that they come from heterogeneous sources in data lakes, some of which are in fact noisy to the model. Therefore, we apply our proposed tuple/feature augmentation techniques [2, 13] to judiciously select tuples/features (from unionable/joinable tables), iteratively integrate to the query table and evaluate the performance. Here, we summarize the high level idea [2] of selecting tuples from unionable tables.

i). *Clustering.* Given all tables in $R(T_U)$, we first merge all tuples of these tables, and then cluster these tuples based on their feature similarities. Then tuples in each cluster can be seen as homogeneous. Next, we use an iterative MAB solution to judiciously select tuples that can benefit the model from these clusters.

ii). *MAB-based data selection.* We take each cluster as a bandit arm. Each arm is assigned with an upper confidence bound (UCB) score,

```
# build index for the datalake
datalake = LakeCompass.DataLake('/datalake_demo')
keyword_index = datalake.metadata.indexing(
    index_type='HNSW', search_type='keyword',
      config=LakeCompass.HNSWconfig())
# keyword based search
candidate_table = LakeCompass.keyword_search(
    keyword_index, 'education')
```

Figure 2: A Code Segment for Index Building and Keyword-based Search of LakeCompass.

which will be updated after each iteration of tuple selection. At the beginning of each iteration, the arm with the highest UCB score will be selected, then a small batch of tuples will be sampled from the cluster and added to train. If the model performance is tested to be improved on a validation set, a reward will be assigned. At a high level, the UCB score is computed by the summation of an exploration and an exploitation score. The less frequently a cluster is picked, the higher the exploration score. The higher reward the cluster has obtained, the higher the exploitation score. Hence, a combination of the two scores used by MAB-based method can well handle the exploration-exploitation trade-off. Specifically, for each cluster C_i, at iteration k:

$$UCB(C_i, k) = R_i^k + \alpha\sqrt{2\ln n^k/(n_i^k+1)} \quad (1)$$

where α is pre-defined, n_i^k is the total number of times that C_i was assigned non-zeros scores from iterations 1 to k, and n^k is the sum of n_i^k for all clusters at iteration k. In Eq. 1, the former part (R_i^k) refers to the *exploitation*, which means to focus more on the cluster where some tuples have resulted in much performance improvement. The latter part ($\sqrt{2\ln n^k/(n_i^k+1)}$) refers to the *exploration*, *i.e.*, focusing more on the clusters that are rarely picked.

iii). *Iterative model evaluation.* We iteratively apply the MAB-based solution until the model performance does not increase within several successive iterations. In order to accelerate the iterative training, we can apply the online learning technique that only trains over the added tuples.

Similarly, the MAB-based solution can also be extended to augment useful features from joinable tables [13].

3 DEMONSTRATION SCENARIOS

This section showcases our demonstration scenarios, which highlight the main functionalities of LakeCompass. These scenarios illustrate how LakeCompass identifies relevant tables within the data lake to enrich tuples or features, thereby enhancing the performance of downstream models. Users can utilize LakeCompass functionalities via Python APIs or Web interfaces.

Datasets. We support index construction, table search and model analysis over 1TB data collected in our benchmark [4] – **100X** larger than the data lakes used in existing works, and the number of columns is up to **100 million**, on which we implemented and evaluated multiple typical table union/join search methods. Moreover, we also provided a sufficient number of labeled diverse

```
# train configuration
predict_col = 'Educational Category'
model_config = {'model': 'SVM', 'type': 'classification',
                'k': 4, 'model_config': SVMconfig()}
query_table = LakeCompass.read('education_Des_Moines.csv')
query_table_model = DownstreamModel(**model_config)
val_set = pd.read_csv('val_set.csv')
test_set = pd.read_csv('test_set.csv')
# trian a model using query table
query_table_model.train(query_table, predict_col, val_set)
query_table_model.eval(test_set, predict_col)
```

Figure 3: A Code Segment for Training Model based on the Query Table.

```
# unionable table search
union_index = datalake.columns.indexing(
    index_type='HNSW', search_type='union',
    config=LakeCompass.HNSWconfig())
unionable_tables = LakeCompass.union_search(
    union_index, query_table)
```

Figure 4: A Code Segment for Unionable Table Search of LakeCompass

queries (more than 10 thousand) – **10X** more than any other works, so as to test the algorithm performance. To this end, we organized a team of 25 graduate students and spent more than *7,500 human hours* on labeling the queries.

(1) **Index Building.** LakeCompass enables the utilization of various index types (including the LSH, HNSW, and inverted index.) to support the typical three table search tasks. As shown in Figure 2, the user builds the HNSW index over the metadata to support keyword-based search. Note that we also support the users to build their own data lakes and build indexes.

(2) **Keyword-based Search.** Users could obtain the training dataset by keyword search. To be specific, as shown in Figure 2, The user could use the Python APIs to set "education" as a keyword and utilize the "keyword_search" function to retrieve relevant tables. Then, tables related to education in different cities of different states in the U.S. are retrieved from our data lake.

(3) **Unionable Table Search.** As shown in Figure 3, users have the flexibility to either upload a query table or choose a table from the keyword-based search as the query table. In such cases, the model based on the solely query table tends to show poor performance. Therefore, they can then employ the "union_search" function to retrieve unionable tables based on semantic similarities. As depicted in Figure 4, users select the "education_Des_Moines.csv" as the query table. Subsequently, additional tables pertaining to education in other cities within the state of Iowa are retrieved through the union search. These retrieved tables share multiple semantically identical columns with the query table, enabling them to be seamlessly unioned together.

(4) **Joinable Table Search.** Users could also interact with the Web

METAM: Goal-Oriented Data Discovery

Sainyam Galhotra, Yue Gong, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {sainyam, yuegong, raulcf}@uchicago.edu

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00213

***Abstract*—Data is a central component of machine learning and causal inference tasks. The availability of large amounts of data from sources such as open data repositories, data lakes and data marketplaces creates an opportunity to augment data and boost those tasks' performance. However, augmentation techniques rely on a user manually discovering and shortlisting useful candidate augmentations. Existing solutions do not leverage the synergy between discovery and augmentation, thus underexploiting data.**

In this paper, we introduce METAM, a novel goal-oriented framework that queries the downstream task with a candidate dataset, forming a feedback loop that automatically steers the discovery and augmentation process. To select candidates efficiently, METAM leverages properties of the: i) data, ii) utility function, and iii) solution set size. We show METAM's theoretical guarantees and demonstrate those empirically on a broad set of tasks. All in all, we demonstrate the promise of goal-oriented data discovery to modern data science applications.

I. INTRODUCTION

Augmenting a dataset by joining it with others can improve the utility of data-driven tasks such as causal inference and supervised machine learning. The abundance of tables in open repositories [1], lakes in organizations [2], and even data markets [3], [4] translates into an abundance of *augmentation candidates*. Identifying good augmentation candidates among many is a data discovery problem. To solve this problem, one could use a traditional data discovery system to identify what tables join with the input data, and then, separately, identify what joins increase the task's utility. This *discover-then-augment* approach works when the discovery system returns candidates relevant to the task. Unfortunately, in practice, it is hard to guarantee the discovery system identifies good augmentations because: i) relevant augmentations depend on the task; and ii) analysts may not know what properties make an augmentation relevant, e.g., what features augment the predictive power of a classifier. The disconnection between data discovery and augmentation presents a research opportunity.

We harness that opportunity with a new approach we call **goal-oriented data discovery**, where data discovery is not treated as a one-time process. Instead, it adapts to the task by performing interventions: a process to augment the initial dataset and validate its utility. *Interventional* queries help to identify augmentations that *cause* an increase in the task's utility, thus steering the discovery process to automatically maximize the task's utility. This achieves two objectives: first, analysts do not need to know what criteria make an augmentation good because the approach is automatic. Second, any downstream task with a utility function benefits from this discovery approach. The consequences of goal-oriented data discovery are significant; consider the following anecdote. We used goal-oriented data discovery to predict "housing prices" in a geographical area. Our technique identified some obvious datasets that a social scientist would have been able to identify using a discovery system, such as "income of people staying in the neighborhood" and "crime stats". But crucially, it also identified non-obvious datasets correlated with housing prices such as "presence of grocery stores" and "number of taxi trips" from those areas. Indeed, many sociologists and economists leverage external data to infer causal relationships between attributes of interest [5], [6], [7]. But they rely on domain knowledge and manual effort to identify those relationships. *Goal-oriented data discovery* paves the way to identify new causal relationships from large data repositories automatically.

A trivial but computationally prohibitive way of solving goal-oriented data discovery is to measure the utility of every augmentation candidate and choose the best. The key technical contributions of our paper focus on developing *interventional querying algorithms* for goal-oriented data discovery that exploit the structure of the data, the utility function, and the solution to adaptively prioritize the candidates for querying.

Interventional Queries for Goal-Oriented Data Discovery. Our technical contributions leverage the following properties to optimize the complexity of discovery without loss of quality.

Properties of the Data. A key insight is that *similar* augmentations perform similarly on the downstream task. We exploit this insight by clustering augmentations and judiciously choosing them from different clusters, thus skipping computation. To cluster augmentations, we represent each with a vector of *data profiles*, which are task-independent measures of data and include semantic similarity, correlation, and mutual information, among others. When a combination of data profiles is correlated with the task's utility, clustering narrows down the number of augmentation candidates.

Properties of the Utility function. A task's optimal utility is given by a set of augmentations. Enumerating all subsets is infeasible. Our insight is that when utility functions are monotonic, i.e., utility never decreases with new augmentations, it is possible to find augmentations efficiently, by considering them one by one. And we can make any function monotonic by ignoring augmentations that harm utility using a wrapper around the user-provided task implementation.

Properties of the Solution set. The optimal set of augmentations is a collection of join paths over different datasets. Most augmentations are irrelevant for a downstream task, and useful augmentations are a small subset of the candidate set. We leverage this property to prioritize small subsets of join

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00213

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

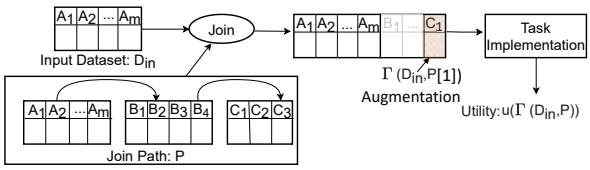


Fig. 1: Notation of a data augmentation pipeline.

paths over larger ones by using combinatorial testing.

METAM: Goal-oriented data discovery We combine the aforementioned properties into an anytime algorithm that finds augmentation candidates by adaptively querying the task. Although goal-oriented data discovery is NP-hard, the algorithm is guaranteed to identify an approximate solution under reasonable assumptions which hold in practice. We also show the efficiency guarantees of the algorithm, and implement it as part of METAM, an end-to-end data discovery approach.

Evaluation Results. METAM automatically finds useful augmentations among millions of them within minutes. We evaluate METAM on large repositories with data from cities in the United States and Kaggle, and for prescriptive analytics tasks such as *what-if* and *how-to* analysis in causal inference, classification and regression in supervised ML, as well as entity linking, and clustering.

Outline. We present notation and discuss the problem in Section II. We present insights for interventional queries in Section III, algorithm in Section IV, and it's analysis in Section V. We present evaluation in Section VI, and related work in Section VII.

II. Goal-Oriented Data Discovery

In this section, we introduce notation and problem statement in Section II-A, and preliminaries in Section II-C.

A. Problem Definition

Figure 1 presents a summary of the notation used in the paper. Let $\mathcal{R}(A_1, \ldots, A_m)$ denote a relation schema over m attributes, where A_i denotes the ith attribute. D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a list of tuples T where each tuple $t \in T$ is a specific instance of the schema. Using this notation, we now define a noisy dataset and a data repository.

Definition 1 (Noisy Structured Data)**.** *A noisy structured dataset D is characterized by incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ denotes missing header values and a list of tuples T which may contain less than m values. Additionally, certain tuples may contain duplicate values.*

Definition 2 (Data Repository)**.** *A data repository contains a set of datasets $\mathcal{D} = \{D_1, \ldots, D_n\}$, where different datasets D_i, D_j may contain overlapping values. Additionally, the datasets may lack schema and contain missing values.*

The different datasets that are joined with the input dataset D_{in} are denoted as a join path, defined below.

Definition 3 (Join Path)**.** *A join path P is defined as an ordered list of noisy datasets $P \equiv \{D_1, \ldots, D_t\}$ such that datasets D_j and D_{j+1} join for all $j < t$ with each other forming a chain of join operations. The dataset formed by joining these t datasets is considered to be augmented with the original dataset D_{in}.*

The augmentation of a join path P to a dataset D_{in} in denoted by $\Gamma(D_{in}, P)$. Augmenting a join path P with D_{in} can augment multiple new columns, some of which may not be required by the downstream task. To distinguish between these different augmented columns, we define augmentation as the projection of the join consisting of a single column.

Definition 4 (Augmentation)**.** *An augmentation $\Gamma(D_{in}, P[j])$ is defined as the jth column that is added after joining the path P with the input dataset D_{in}.*

Note that $\Gamma(D_{in}, P) \equiv \{\Gamma(D_{in}, P[j]), \forall j \in \{1, 2, \ldots, l\}\}$, where l is the number of columns after materializing the join-path P. Γ considers materializing a join path to add a new attribute. Other types of data augmentation include unions (addition of additional rows), relational embedding based augmentations, spatio-temporal augmentations, etc. Empirically, we observed that join-path based addition of new attributes is commonly used for popular applications. However, our framework extends to any general set of augmentations. A task t is a black box consisting of the analysis algorithm that takes a dataset (D_{in} or an augmented version of D_{in}) as input and outputs the performance measure of the task. We call "query" the process of obtaining a task's utility on a dataset. We formally define the utility score of the task as follows.

Definition 5 (Utility score)**.** *Given a task t that operates on a dataset D, the utility score $u_t(D)$ is defined as the objective (or the evaluation metric) of the task when operated with D.*

Without loss of generality, we assume that the utility score is normalized $u_t(\cdot) \in [0, 1]$ and that higher utility score means better task quality. Different sets of augmentations may improve the task's utility. We seek a *minimal* set that contains only the augmentations that help to maintain high utility.

Definition 6 (Minimal set of Augmentations)**.** *Given an initial dataset D_{in}, a mechanism to compute $u_t(\cdot)$, a set of augmentations $\mathcal{P} = \{P_1[i_1], \ldots, P_j[i_j]\}$ is considered minimal if removing any attribute from $\Gamma(D_{in}, \mathcal{P})$ reduces task utility.*

Problem II.1 (Goal-oriented discovery)**.** *Given an initial dataset D_{in}, a task t that has a mechanism to compute $u(D')$ for any table D', the problem of identifying augmentations that optimize the task is to identify a minimum augmentation set $\mathcal{T}$ such that the augmented dataset has utility $u(\Gamma(D_{in}, \mathcal{T})) \geq \theta$.*

B. Task implementation and utility calculation

A task takes a dataset as input and returns a utility score that depends, at least in part, on the input dataset. Some examples:

1) Predictive analytics: The task trains a machine learning model over an input training dataset. The training code could be simple, e.g., a random forest, or complex, such as an AutoML solution. The utility correspond to a model-relevant metric such as accuracy, F1 score and may also include fairness and robustness metrics.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

2) Prescriptive analytics involves hypothetical analysis (what-if and how-to queries), causal inference, and explainability as the key components [8], [9], [10], [11]. The utility corresponds to a metric that summarizes the quality of generated explanations, support of identified causal dependencies, and total causal effect of attributes.
3) Data preparation and visualization: These include entity resolution, entity linking, data cleaning and visualization. The utility often corresponds to F-score and accuracy.

Note that these are some examples of task implementation and any advances in feature engineering, data transformation and data analysis can be incorporated in the utility function.

C. Preliminaries and Problem Discussion

We use previous data discovery techniques [12], [13], [14], [15] to obtain a set of augmentations from large data repositories; we use Aurum [12]. Although existing solutions may generate noisy candidates (due to the use of approximate techniques and semantic ambiguity), our approach works even when different sources of noise are present. We also leverage existing profiling techniques [16], [17], [18], [19] to describe the augmentation candidates $\Gamma(D_{in}, \mathcal{P})$. For example, a data profile $\langle Corr, C_1, C_2 \rangle$ refers to the correlation between columns C_1 and C_2. We use the term *data profile value* to denote the value of the characteristic/property. A data profile may refer to a single attribute (e.g. domain of an attribute) or multiple attributes (correlation between attributes), and these properties may be correlated with downstream utility.

Definition 7 (Data Profile)**.** *A data profile X is defined as the property of a dataset D such that the tuples in D satisfy X.*

We consider the following data profiles in our implementation.
• **Correlation and Mutual Information (MI)**: MI is often used to evaluate causal relationship between attributes [20]. Correlation and MI are also predictors of the quality of attributes in machine learning tasks. This profile estimates the Pearson correlation (MI) of the candidate augmentation P and the attributes of D_{in}.
• **Semantic-embedding based distance** profile captures the semantic similarity between the considered datasets. This profile is computed as the cosine similarity between the embeddings of both datasets constructed from pre-trained models such as BERT [21]. The dataset embedding is constructed by averaging the embedding vectors of tokens present in the table.
• **Dataset metadata/attributes** profile calculates the similarity between datasets based on their source and attributes. Unlike the semantic embeddings, this profile captures syntactic similarity between attributes and dataset source, which is commonly used to estimate the quality of augmentations [22]
• **Dataset overlap** profile calculates the cardinality of the final dataset identified after augmentation.

Extending to other data profiles. Metam can be extended with new profiles to cater for new downstream tasks (e.g., anomaly detection [23], conditional independence for fairness [24]) or to leverage advances in profiling techniques [23], [25], [26], [27].

Problem Discussion: We discuss how various discovery-then-augment baselines fail to solve the problem efficiently:
• **Join Everything**: This technique joins D_{in} with every join path identified. This approach may bring irrelevant attributes that deteriorate the utility. Furthermore, the approach is infeasible in large-scale scenarios with thousands of augmentations.
• **Uniform sampling**: This technique samples join paths out of the identified joinable datasets uniformly at random and uses them to augment D_{in}. This approach does not guarantee that the chosen sample will improve the utility score.
• **Join Path overlap ranking**: A common technique is to rank join paths based on the cardinality of the augmented datasets (used by S4 [14] and Ver [22]). This technique identifies datasets that contain fewer missing values, but does not guarantee to optimize the task.
• **Using data profiles for selection**: Analysts may use intuition to rank augmentations based on profiles that are expected to be useful. This approach may work if the profiles are accurate and the domain scientist's intuition is correct. But this approach will not work well otherwise, and it relies on accurate estimation of profiles, which are hard to compute in large-scale data repositories, where approximate techniques are often necessary for scalability.

The crux of the problem is that these baselines are unaware of the task, so there is no guarantee that the chosen augmentations will improve utility. Solving goal-oriented data discovery requires an interventional approach that seeks to understand what augmentations *cause* the task's utility to increase.

III. Intervention-based Querying

In this section, we first describe the limitations of baseline techniques and then discuss how the properties of data, utility function, and solution let us design an efficient technique.

A. Baseline Interventional Techniques

Goal-oriented data discovery can be solved by enumerating every subset of candidate augmentations, computing their utility, and choosing the minimal subset that achieves utility of at least θ. With n candidate augmentations, this process may require up to $O(2^n)$ queries (does not finish for $n > 30$), so this approach is unfeasible. We consider two improvements:
• **Utility-based selection:** Given a ranking of augmentations (e.g., based on some data profiles) this solution queries, iteratively, in ranking order until the utility obtained is higher than the desired threshold, θ. This solution will be inefficient every time the data profile is not related to the task. And choosing a data profile related to the downstream task is non-obvious for many of the interesting data-driven tasks considered.
• **Prediction from expert advice:** To avoid selecting a profile a priori, different data profiles can be treated as experts and we can use expert selection techniques to solve the problem. For example, the multiplicative weights update method (MW) [28] estimates the ability of different experts to improve utility. MW guarantees the selection of the best expert in hindsight. In its simplest form, considering each profile as a different expert fails to identify combinations of profiles that best

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:59:06 UTC from IEEE Xplore. Restrictions apply.

Selecting Sub-tables for Data Exploration

Yael Amsterdamer
Bar-Ilan University
amstery@cs.biu.ac.il

Susan B. Davidson
University of Pennsylvania
susan@cis.upenn.edu

Tova Milo
Tel Aviv University
milo@post.tau.ac.il

Kathy Razmadze
Tel Aviv University
kathyr@mail.tau.ac.il

Amit Somech
Bar-Ilan University
somecha@cs.biu.ac.il

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00192

***Abstract*—Data scientists frequently examine the raw content of large tables when exploring an unknown dataset. In such cases, small subsets of the full tables (*sub-tables*) that accurately capture table contents are useful. We present a framework which, given a large data table T, creates a sub-table of small, fixed dimensions by selecting a subset of T's rows and projecting them over a subset of T's columns. The question is: Which rows and columns should be selected to yield an *informative* sub-table?**

Our first contribution is an informativeness metric for sub-tables with two complementary dimensions: *cell coverage*, which measures how well the sub-table captures prominent data patterns in T, and *diversity*. We use association rules as the patterns captured by sub-tables, and show that computing optimal sub-tables directly using this metric is infeasible. We then develop an efficient algorithm that indirectly accounts for association rules using *table embedding*. The resulting framework produces sub-tables for the full table as well as for the results of queries over the table, enabling the user to quickly understand results and determine subsequent queries. Experimental results show that high-quality sub-tables can be efficiently computed,and verify the soundness of our metrics as well as the usefulness of selected sub-tables through user studies.

***Index Terms*—Interactive data exploration and discovery**

I. Introduction

Data exploration is the process of understanding an unfamiliar dataset and determining what part of the data is relevant to an analysis task. The process is often done by iteratively applying exploratory queries, examining their results, and interpreting them with data visualizations.

Numerous lines of work have studied solutions for different aspects of the exploration process, such as query suggestions (e.g., [1], [2]), query formulation for non-programmers [3]–[6], insights-discovery [7]–[9], and visualization recommendations [10]–[12].

In this work, we focus on the task of *tabular data display*: before applying additional queries or visualizations, users often manually examine the raw table or query results. This is done using commands such as the table display of the popular Pandas library[1], which shows a small subset of the raw table rows and columns that is easier for a human user to inspect than the full table. However, a disadvantage of the Pandas tabular display is that its choice of rows and columns to display is arbitrary: it includes a few of the first and last rows and columns. As a result, the sub-tables may include arbitrary data values and cross-column value combinations rather than ones that reflect prominent trends; it may also elide columns that are important for further exploration and analysis.

[1]Pandas: Python Data Analysis Library. https://Pandas.pydata.org/

Example I.1. *Consider a table T taken from the Kaggle* flights *dataset [13] which contains 31 columns and ∼6M rows. T may be used for analysis tasks such as predicting flight delays and cancellations.*

The analyst starts by inspecting the data using Pandas display(T)*, which yields the sub-table displayed at the top left of Figure 1.*

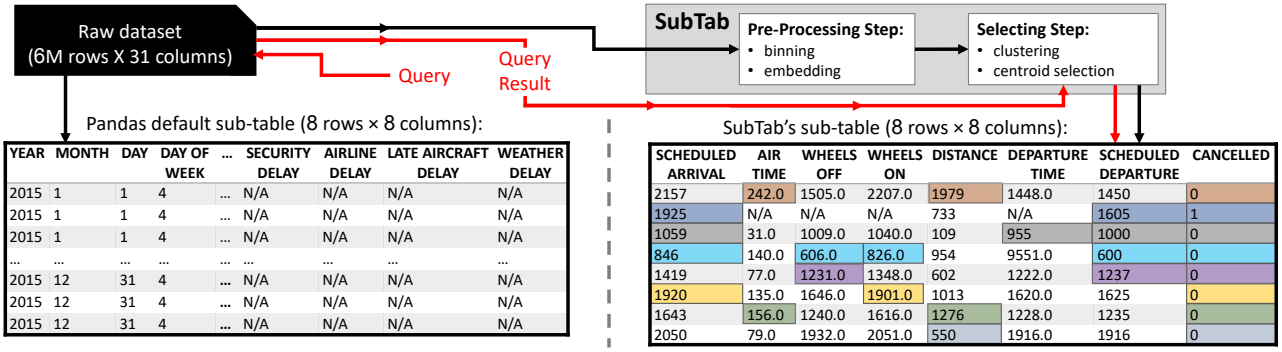
The usefulness of this display for data exploration is intuitively limited: for instance, the first rows, last rows, and last columns are repetitive, and the displayed values are arbitrary. In addition, the chosen columns may not be relevant for the analyst's goals: for instance, the analyst may be interested in flight cancellations, yet there is no reason to assume that the columns in the Pandas *sub-table are relevant for this goal.* □

Our goal is to develop a foundational approach for an informed selection of sub-tables that characterizes a given table as a whole and captures multiple diverse patterns. Concretely, given a table T with n rows and m columns, our goal is to create a sub-table T_{sub} with $k << n$ rows and $l << m$ columns which is a subset of k rows projected over a subset of l columns of T. To do this, we introduce a novel notion of sub-table informativeness which reflects (1) the extent to which the sub-table captures prominent *data patterns* and trends in the full table, including cross-column patterns. This is based on *association rules* (ARs) as a standard type of pattern that applies to a subset of a row's cells and hence allows several such patterns to be captured is a small sub-table. (See a discussion of other types of patterns in Section VII.); (2) the *diversity* in the sub-table, including representative values from each selected column and minimizing data repetitions. We also allow users to specify *target columns* that affect the choice of sub-tables so that it captures columns and ARs relevant to the target column.

Example I.2. *Continuing with the* flights *dataset, an* informative *sub-table is shown at the right of Figure 1. Unlike the sub-table at the top left, this sub-table captures several prominent ARs that hold over the input table and that include the target column* CANCELLED*. For purposes of illustration, we show this by highlighting cells that participate in one of the ARs that hold for the row (many additional ARs hold). For example, the highlighted rule in the first row states that long flights (*AIR_TIME$\in [198.0, 422.0]$ *and* DISTANCE$\in [1546.0, 2724.0]$*) are likely* not *to be cancelled (*CANCELLED$= 0$*). The highlighted rule in the second row states that short afternoon flights (according to the* SCHEDULED_DEPARTURE *and* SCHED-

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00192

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.



YEAR	MONTH	DAY	DAY OF WEEK	...	SECURITY DELAY	AIRLINE DELAY	LATE AIRCRAFT DELAY	WEATHER DELAY
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
2015	1	1	4	...	N/A	N/A	N/A	N/A
...	...	...	...	...	...	...	...	...
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A
2015	12	31	4	...	N/A	N/A	N/A	N/A

SCHEDULED ARRIVAL	AIR TIME	WHEELS OFF	WHEELS ON	DISTANCE	DEPARTURE TIME	SCHEDULED DEPARTURE	CANCELLED
2157	242.0	1505.0	2207.0	1979	1448.0	1450	0
1925	N/A	N/A	N/A	733	N/A	1605	1
1059	31.0	1009.0	1040.0	109	955	1000	0
846	140.0	606.0	826.0	954	9551.0	600	0
1419	77.0	1231.0	1348.0	602	1222.0	1237	0
1920	135.0	1646.0	1901.0	1013	1620.0	1625	0
1643	156.0	1240.0	1616.0	1276	1228.0	1235	0
2050	79.0	1932.0	2051.0	550	1916.0	1916	0

Fig. 1: System Workflow. SubTab takes as input a raw table used throughout the EDA session. The sub-table on the left is the output of the display command in Pandas, a random subset of the table's rows and columns. The sub-table on the right is the output of SubTab, in which some of the ARs are highlighted for purposes of illustration. SubTab has two-steps: (1) Pre-Processing Step - the table is binned and embedded into vectors; (2) Selecting Step - triggered by a display of the input table or as the result of a query over the table, an *informative sub-table* is displayed. This is done by clustering the row and column vectors of the query result in the embedded space and selecting the centroids as representatives.

ULED_ARRIVAL columns) are likely to be canceled. The sub-table also shows diverse values per column by including representatives of diverse sub-groups in the value distribution, e.g., morning, afternoon and night arrivals/departures, short, medium and long flight distances, etc. □

We formalize (Section II) our notion of informativeness based on a combination of two complementary metrics: *cell coverage* and *diversity*. Cell coverage measures how well the sub-table T_{sub} represents data patterns in the full table T, which are defined via ARs.

Concretely, given a set of prominent ARs that hold over the full table T,[2] our metric reflects the number of cells in T that are describable by ARs that are captured in T_{sub}. If one or more *target columns* are known in advance to be the focus of the analysis, they will be included in the l selected columns, and we measure cell coverage only according to ARs that include one or more target columns. For the second metric, *diversity*, we split the value distribution of a column (continuous or categorical) into meaningful *bins*. We then compute a Jaccard-like row similarity score based on these bins, and use the average pairwise row distance as the diversity score. Intuitively, in a small sub-table we cannot capture the full distribution of each column, and our metric is geared towards increasing the number of represented bins and reducing bin repetitions. We use a score combining these metrics to measure the informativeness of a sub-table.

Unfortunately, we show (Section III) that optimizing our informativeness measures directly is generally infeasible, and discuss the limitations of approximate solutions. In particular, although there are several efficient techniques for mining ARs (e.g., [14]–[18]), they are still too time-consuming for large datasets in an interactive setting.

We therefore propose a sub-table selection method which *indirectly* accounts for ARs using *table embedding* [19], [20], and that allows us to avoid calculating ARs thereby speeding up and simplifying the entire process (Section IV). Concretely, given a table T we use *binning* [15] to split each column's values into a small set of meaningful groups. We then compute an embedding of table cells as vectors. As we demonstrate in Section IV-B, the embedding captures bin co-occurrences, and therefore implicitly corresponds to ARs. To select rows and columns for a sub-table we derive from the cell vectors a vector representation for rows and columns, cluster them (separately), and select the centroids as rows and columns that represent diverse patterns in the data.

An important benefit of our solution design is in *responding to queries over* T: during the exploratory data analysis (EDA) session, users typically issue different queries on a given table T (red arrows of Figure 1). Our computation of cell embedding may be viewed as a part of the pre-processing step of a given data table T, along with the binning of its values (first blue box in Figure 1). This step is executed once upon loading the table. Then, for subsequent selection-projection query Q on T, we can compute the vector representation of rows and columns in $Q(T)$ based on the cells of T that appear in them, and re-execute clustering and centroid selection (*Selecting* step in Figure 1, shown as the second blue box). This significantly speeds up sub-table computation compared with computing everything from scratch (a few seconds instead of up to a minute for large tables).

Contributions: Main contributions of this work include:

1) A formal notion of *informativeness* for sub-tables, designed to measure how well a sub-table characterizes the full table. The capturing of prominent data patterns in the sub-table is measured by *cell coverage* (for ARs), while the ratio of non-repetitive, representative values is measured by *diversity*.

2) A *complexity analysis* of the problem of selecting an optimal sub-table, which shows the infeasibility of computing sub-tables that optimize our informativeness metrics.

3) A *greedy algorithm* (CCSG), which traverses column combinations, and greedily selects rows for each such combination. This algorithm approximates the optimal cell coverage if all column combinations can be traversed within a given

[2]There are standard metrics we can use to measure the prominence of ARs in T, such as Support and Confidence [14]. Also see Section V-D.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.

time limit. We also present RCCSG, a variant that samples rows to speed up row selection. We discuss the practical limitations of both algorithms, and use them as baselines against which we compare the quality of computed sub-tables.

4) A *practical algorithm*, SubTab, for computing informative sub-tables, which accounts for ARs indirectly using table embedding. The algorithm has two phases: a pre-processing phase that performs binning and embedding, and can be executed as soon as the data is loaded; and a clustering and centroid selection phase that is called for each sub-table display, e.g., on query results.

5) An *implementation* for SubTab as a local Python library that hooks into Pandas, and displays tables and query results as informative sub-tables. Its UI includes optional highlighting of ARs (as shown in Figure 2).

6) *Experimental results* that measure the sub-table quality and running time of SubTab compared with several baselines. Among the interactive time algorithms, SubTab achieves the best quality by a large gap. Among the algorithms that achieve the best quality, including ones that directly optimize our metric, SubTab is the only one which executes in interactive speed. Experiments with real EDA sessions and user studies show that our quality metrics are compatible with both subjective user experience and objective performance in insight discovery.

Organization: The rest of the paper is organized as follows. We start in Section II by defining our metrics of cell coverage and diversity. Section III gives hardness results for the problem of optimal sub-table selection and greedy algorithms. Section IV presents our sub-table computation method based on table embeddings. Section V describes our experimental study. Related work is discussed in Section VI. We conclude in Section VII.

II. Model and Metrics

In this section, we formalize the notion of "sub-table informativeness" via metrics of cell coverage and diversity and define an optimization problem based on their combination.

A. Model

A relational schema $U = \{u_1, \ldots, u_{|U|}\}$ is a finite set of columns, such that each column u_i allows values from a subset of the global domain $\mathcal{D}_i \subseteq \mathcal{D}$ (e.g., for a binary column, $\mathcal{D}_i = \{0, 1\}$). A relational table over U is a finite set $T \subseteq U \to \mathcal{D}$ of tuples such that $t(u_i) \in \mathcal{D}_i$ is the value of the cell in the row corresponding to a tuple $t \in T$ and the column $u_i \in U$.

Definition II.1 (Sub-table)**.** *Given a table T over schema U, a sub-table T_{sub} is a table over schema $U_{sub} \subseteq U$ such that each tuple $t \in T_{sub}$ is the projection of some tuple $t' \in T$ over the columns of U_{sub}, i.e., for every $u \in U_{sub}$, $t(u) = t'(u)$.*

We next define two standard notions that will be useful in the sequel: binning and ARs.

In a schema U, each column u_i may be *categorical*, namely, $\mathcal{D}_i$ is discrete, e.g., a column of airline names; or *continuous*, namely, $\mathcal{D}_i$ is a continuous range, e.g., a column of flight distance. Moreover, in a table T over U a different distribution

```
from SubTab import subTab
sdf = subTab(flights_df, use_rules=True)
df1 = flights_df[(flights_df['DISTANCE']>2000) & (flights_df['DEPARTURE_DELAY']>10)]
sdf.display(df1)
```

For creating new table vectors, it took 0:00:02.282
For summary creation, it took 0:00:00.45
{'cell_cov': 0.31. 'jaccard':0.72}

Destination Airport	Arrival Time	Wheels On	Airline Delay	Departure Time	Wheels Off	Scheduled Departure	Late Aircraft Delay	Distance	Departure Delay
MCO	1842.0	1832.0	102.0	1050.0	1107.0	855	0.0	2218	115.0
JFK	742.0	736.0	N/A	2333.0	2345.0	2303	N/A	2475	28.0
Rule #2: {(Destination Airport -'SEA'), (Arrival Time -'high'), (Distance -'high')} →{(Airline Delay -'low'), (Departure Delay -'high')}, support-0.11, confidence-0.5, lift-1.5									
SEA	2300.0	2253.0	20.0	1939.0	2012.0	1807	72.0	2402	92.0
SAN	1941.0	1938.0	N/A	1649.0	1702.0	1629	N/A	2588	20.0
BOS	1550.0	1545.0	N/A	752.0	806.0	730	N/A	2611	22.0
SFO	2136.0	2130.0	N/A	1820.0	1834.0	1808	N/A	2704	12.0
JFK	16.0	3.0	0.0	1603.0	1615.0	1535	17.0	2475	28.0

Fig. 2: Informative 10X10 sub-table for a large query result

of values (e.g. uniform or skewed) may occur in each column. *Binning* the column values is a technique commonly used to allow a uniform treatment of columns with different ranges and distributions. Formally,

Definition II.2 (Binning)**.** *Given a table T over schema U, a* binning *function $\mathcal{B}$ maps each column $u_i \in U$ to a finite set of bins $\mathcal{B}_i = \{B_1^i, \ldots, B_{|\mathcal{B}_i|}^i\}$ such that for every $t \in T$, $t(u_i)$ belongs to exactly one bin $B_j^i \in \mathcal{B}_i$.*

Example II.3. *In the flights' dataset, we split the (continuous) range of the DISTANCE column into the bins short, medium, and long-distance. Depending on the column value distribution, we may obtain $B_{long}^{DIST} = [1546.0, 2724.0]$. The CANC. column is binary, hence we can use its categories as bins.*

Next, we recall the notion of *association rules* (ARs), which we use to capture patterns in the data. ARs will be used to measure and compare the quality of sub-tables. Formally,

Definition II.4 (Association rules (ARs) [15])**.** *Given a table T over schema U, an AR R has the form $\{(u_1, v_1), \ldots (u_r, v_r)\} \rightarrow \{(u_{r+1}, v_{r+1}), \ldots (u_{r+p}, v_{r+p})\}$ where each $u_i \in U$ is a column and each $v_i \in \mathcal{D}_i$ is a cell value. Denote by $U_R = \{u_1, \ldots, u_{r+p}\} \subseteq U$ the set of columns used in R. We say R* holds *for a tuple $t \in T$ if $t(u_i) = v_i$ for every $1 \leq i \leq r + p$. Denote by $T_R \subseteq T$ the subset of tuples for which R holds.*

The use of binning may improve mined ARs [15]: given a table T, we can replace each cell value $t(u_i)$ with an identifier of its matching bin B_j^i. Consequently, one may be able to mine ARs that apply to more tuples.

Example II.5. *Using the bins from example II.3, the AR from Example I.2 stating that long flights are likely* not *to be cancelled can be written as:*
AIR_TIME$\in B_{long}^{AT}$,*DISTANCE*$\in B_{long}^{DIST} \nrightarrow$*CANC.*$\in B_0^{CANC}$.

B. Informativeness Metrics

We now develop quality metrics for sub-tables. Out of various possible informativeness metrics, we base our choice on the role of a sub-table as a compact view of the raw data that characterizes the table as a whole. The first type of metric

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:35 UTC from IEEE Xplore. Restrictions apply.

Ver: View Discovery in the Wild

Yue Gong*, Zhiru Zhu*, Sainyam Galhotra, Raul Castro Fernandez
Department of Computer Science, The University of Chicago
Email: {yuegong, zhiru, sainyam, raulcf}@uchicago.edu
*Yue Gong and Zhiru Zhu contributed equally

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00045

***Abstract*—We present `Ver`[1], a data discovery system that identifies project-join views over large repositories of tables that do not contain join path information, and even when input queries are inaccurate. `Ver` implements a reference architecture to solve both the technical (scale and search) and human (semantic ambiguity, navigating a large number of results) problems of view discovery. We demonstrate users find the view they want when using `Ver` with a user study and we demonstrate its performance with large-scale end-to-end experiments on real-world datasets containing tens of millions of join paths.**

I. Introduction

The existence of large repositories of data such as those that originate from the combination of disparate databases [1], data lakes [2], open data portals [3], and cloud repositories [4] has the upside of offering opportunities to find valuable data for tasks such as machine learning, reporting, and data analytics. The downside is the resulting data discovery problem: identifying a combination of datasets useful for the downstream task even when these reside in different databases. For example, a machine learning engineer may need a training dataset that requires combining a table in a database with the one sitting on the enterprise data lake. Large volumes of often incomplete and noisy data without any join information, which we call *pathless table collections*, make solving data discovery difficult and thus hampers productivity.

There are several approaches to identify project-join views (PJ-views) over pathless table collections. Discovery systems such as Aurum [5], Goods [6], Auctus [7], Juneau [8], Josie [9], Table-Union [10], D3L [11] and libraries such as LSHEnsemble [12], and Lazo [13] help with identifying datasets that satisfy some relevance criteria whether via keywords [6] or programs [5]. Analysts can then combine the datasets to verify that it satisfies the view they need. Another approach is query-by-example (QBE) interfaces [14], [15] that lets users provide examples of the view they need. These can be adapted, with effort, to run on top of data discovery systems. Whether via keywords, programs, or QBE interfaces, the result of discovery queries over pathless table collections leads to ambiguous results that include semantically distinct results, duplicates or near-duplicates, and different versions of the data. This ambiguity further complicates identifying the right view. More generally, solving view discovery in the wild requires addressing the following challenges:

- **Challenge 1. Noisy Queries.** Users provide queries that represent their best knowledge of the data in the form of keywords, programs, and examples. User-provided input may or may not appear in the table collection and they may be noisy and incorrect.
- **Challenge 2. Noisy Join Paths.** Pathless table collections do not include join paths. Identifying true join paths automatically is impossible. We resort to identifying inclusion dependencies, a proxy to join paths, i.e., a join path implies an inclusion dependency but not the other way around.
- **Challenge 3. Large Number of Join Paths.** Large volumes of data result in a large number of join paths that require efficient data structures to represent and navigate them.
- **Challenge 4. Noisy Result Views.** A large number of join paths implies many views may satisfy a user query. Such result views will be noisy due to ambiguity in the user query and noisy join paths. Concretely, there will be duplicate result views, views that are contained within each other, others that are complementary, and others that will show contradictory values for the same key.
- **Challenge 5. Result View Navigation.** Result views will contain semantically ambiguous results, e.g., views with "work address" and "home address". Only users know the right context so the challenge is to elicit that context and use it to choose the view they need among all result views.

In this paper, we introduce a *reference architecture to identify PJ-views in the wild*. Discovering a PJ-view over pathless table collections requires understanding human preferences and requirements (which we refer to as a *human* problem) and solving a *technical* problem. The architecture divides the larger problem into smaller ones, each of which we tackle with a different component. Reference architectures help conceptualize problems and have been influential in advancing the field. For example, [16], [17] for relational databases and [18], [19], [20], [21] for data integration. The reference architecture we propose tackles the five challenges above. While challenges 1-3 are addressed by existing work, we introduce new techniques to address Challenge 4 and 5 in this paper, and demonstrate them as part of an implementation of the reference architecture. We make the following contributions:

- An end-to-end system, **`Ver`**, that identifies PJ-views among tens of millions of join paths. **`Ver`** implements the reference architecture for QBE-based interfaces. We choose QBE because it permits users to declare the table they would like to find, even when they do not know examples and can only provide attribute names (Section III). **`Ver`** relies on existing work [5] to address Challenges 1-3.

[1]code is available at https://github.com/TheDataStation/ver

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00045

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.

- A **view presentation** component that helps humans to identify a good view among many. The approach uses a novel bandit-based approach to *learn* user-specific discovery preferences, and it uses abduction-based reasoning to quickly narrow down the search space, reducing the number of human interactions (Section IV).
- A **view distillation** component that automatically filters out views by classifying them into **4C categories**: *compatible*, *contained*, *complementary*, and *contradictory*. Besides helping with filtering, these categories produce information used by the view presentation component (Section V).

We conduct an IRB-approved user study (Institutional Review Board) [22] to validate **Ver**'s approach to the human problem. We conduct thorough experiments on queries from open data repositories that lack join paths.

II. Definitions and Problem Statement

$\mathcal{R}(A_1, \ldots, A_m)$ is a relation schema over m attributes, where A_i denotes the ith attribute. A table D comprises a schema $\mathcal{R}(A_1, \ldots, A_m)$ and a set of tuples T where each tuple $t \in T$ is a specific instance of the schema.

In practice, tables do not look like the ideal defined above because they may *lack header information*, have *ambiguous names* and contain *dirty and noisy data*. More formally:

Definition 1 (Noisy structured data): A noisy data D is characterized by an incomplete schema information $\mathcal{R}(A_1, \ldots, A_m)$ where $A_i = \phi$ for missing header values and tuples T such that each tuple $t \in T$ contains at most m values.

In addition, a pathless collection may contain tables with contradictory values, e.g., two census tables with different population counts for the same states of the country. Formally, two tables D_i and D_j contradict if the tables contain different values for the same key. We discuss the detection of contradictions in Section V.

Definition 2 (Pathless table collection): A pathless table collection contains a set of noisy tables $\mathcal{D} = \{D_1, \ldots, D_n\}$ where each D_i is a noisy table and tables D_i, D_j may contain contradictory values.

PJ-example-query. A PJ-query (hereafter called query) contains (possibly noisy) example tuples of the desired output. The examples are a proxy to user's discovery requirements. The quality of examples depends on user's knowledge. Given a query q, there may be many tables that contain the input examples, and many combinations of these may satisfy q, resulting in a large number of candidate PJ-views.

Definition 3 (Noisy query): An input query q is a noisy table consisting of l example tuples, $\chi = \{\chi_1, \ldots, \chi_l\}$ where each χ_i is a noisy tuple denoting example values that are expected to be present in the desired output. The different columns in the examples χ are denoted by $\chi.A_i, \forall i \in \{1, \ldots, \tau\}$, where τ is the number of attributes in the input query.

PJ-views are constructed by joining datasets through keys. We first define a join path and then use it to discuss the effects of noise, followed by a formal problem statement.

Definition 4 (Join path): A join path P is defined as an ordered set of noisy tables $P \equiv \{D_1, \ldots, D_t\}$ such that tables D_j and D_{j+1} join for all $j < t$ via a key column $k \in D_j, D_{j+1}$ forming a chain of join operations.

Joinable datasets can be identified in the presence of key information, which is generally absent in pathless scenarios. A PJ-view V is the dataset formed after materializing the path P followed by projection, i.e., choosing the relevant columns.

Problem 1 (Project-Join view discovery over pathless table collections): Consider a pathless table collection $\mathcal{D}$ and a query q with examples χ. The goal is to construct a minimal candidate set of PJ-views $\mathcal{D}'$ that satisfy the user requirements.

Users may require the PJ-view to contain all examples χ, or any of them, depending on user and application. Formalizing these requirements is outside the scope of this work.

III. A Reference Architecture

Why a Reference Architecture? From a systems engineering perspective, a reference architecture is the materialization of a "divide and conquer" strategy that splits complex engineering problems into smaller parts. Thus, a reference architecture describes a collection of components and their interactions. In making a reference architecture concrete, we state our understanding of the problem and represent it as a concrete artifact that the community can scrutinize and improve.

Design Overview

Finding a PJ-view in the wild requires solving a *human* and a *technical* problem. We present a reference architecture for view discovery that contains components targeting both *human* and *technical* problems. Along with each component, we discuss implementation options, including those made by **Ver**. We also briefly discuss two novel components, View Distillation and View Presentation.

Overview. Algorithm 1 shows pseudocode (human components are highlighted in blue) and Fig. 1 shows **Ver**'s architecture inside a funnel, denoting the gradual reduction of views as data flows downstream. **Ver** builds a discovery index offline.

Discovery Engine and Index Creation (technical). *This component builds indices over pathless table collections: i) a join path index, which can be approximate; ii) retrieval indices over table names, values, attribute names and column similarity. The indices are available online, via the Engine's API to other components. (**Challenge 2**).*

The indices can be built using state-of-the-art methods such as Aurum [5] (that **Ver**'s implementation uses), Auctus [7], JOSIE [9], LSHEnsemble [12]. After the indices are built, users design and submit queries via a VIEW-SPECIFICATION component (line 1).

View Specification (human). Discovery interfaces include spreadsheet-style, keyword search, APIs, natural language, and combinations of these. The reference architecture supports these interfaces via the View-Specification component. For QBE-based interfaces, as implemented by **Ver**, the input is a set of examples, χ, and the output of this stage is a set of example attributes and values. View-Specification

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: **Ver** Design Overview

Input : Pathless table collection $\mathcal{D}$, Discovery Index I
Output: PJ-views $\mathcal{V}$

1 $\chi \leftarrow$ VIEW-SPECIFICATION($\mathcal{D}$)
2 CAND $\leftarrow \phi$
3 **for** $\chi.A_i \in$ COLUMNS(χ) **do**
4 CAND(A_i) $\leftarrow$ COLUMN-SELECTION($\chi.A_i, \mathcal{D}, I$)
5 **if** MODE==*Interactive* **then**
6 CAND(A_i) $\leftarrow$ Query CAND(A_i)
7 CAND $\leftarrow$ CAND $\cup$ CAND(A_i)
8 $\mathcal{V}_{PJ} \leftarrow$ JOIN-GRAPH-SEARCH(CAND, χ)
9 $\mathcal{S} \leftarrow$ VIEW-DISTILLATION($\mathcal{V}_{PJ}$)
10 **if** MODE==*Interactive* **then**
11 $\mathcal{V} \leftarrow$ VIEW-PRESENTATION($\mathcal{V}_{PJ}, \mathcal{S}$)
12 **else**
13 $\mathcal{V} \leftarrow$ Rank $\mathcal{V}_{PJ}$ based on overlap score
14 **return** $\mathcal{V}$

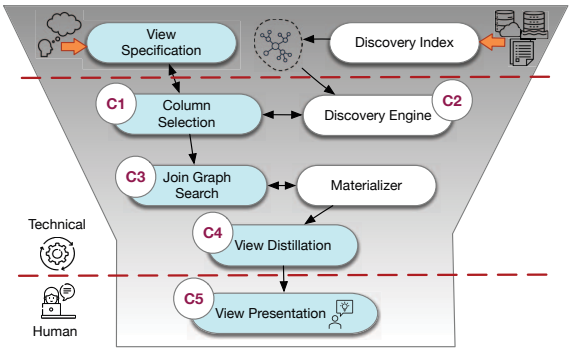


Fig. 1: **Ver** Reference Architecture. The funnel illustrates the progressive reduction of data as it flows downstream.

may interact with users when it detects the provided examples are ambiguous, or to offer examples to choose from.

Next, the COLUMN-SELECTION component selects the subset of tables containing user-provided examples, χ (lines 3-7).

COLUMN SELECTION (TECHNICAL). *This component must be designed to identify relevant data even when the input query is noisy, addressing* ***Challenge 1*** *above. The output of this component is a collection of candidate tables and columns.*

The candidate columns are processed by the JOIN-GRAPH-SEARCH component (line 8) to enumerate and materialize candidate PJ-views (addresses Challenge 3). The candidate view search space reduces, as shown in Fig. 1.

JOIN GRAPH SEARCH (TECHNICAL). *Given a set of candidate tables, an input query, and the discovery index providing join paths, this component identifies all* join graphs *that, when materialized, produce candidate PJ-views. The main goal of this component is to address the large join path space (****Challenge 3****). To materialize candidate PJ-views, the* JOIN-GRAPH-SEARCH *component uses a* MATERIALIZER*, a data processing component with the capacity to execute PJ queries.*

JOIN-GRAPH-SEARCH returns many *candidate views*. Ranking the views is hard because of users' differing search criteria. The VIEW-DISTILLATION component (line 9) summarizes the candidate views, further reducing the view search space as shown in Fig. 1.

VIEW DISTILLATION (TECHNICAL). *This component computes* categories *from the candidate PJ-views that include redundancy and containment in the views, as well as opportunities for unioning views and more. Some categories can be used to distill/summarize the views (****Challenge 4****). Others are shared with the downstream component.*

VIEW-PRESENTATION receives the distilled views. It can *rank* the views and return top-1 (for a full automated mode) or it can leverage the categories computed by VIEW-DISTILLATION to help users find the right data (lines 10-13).

VIEW PRESENTATION (HUMAN). VIEW-PRESENTATION *uses different question interfaces to elicit information from users via data questions. The questions are designed to narrow down the space until users find the desired view (****Challenge 5****). The component chooses what questions to ask, sequentially, using a bandit-based approach.*

Next, we present VIEW-PRESENTATION (Section IV), VIEW-DISTILLATION (Section V).

IV. VIEW PRESENTATION

Ideally, a query results in one PJ-view. In practice, ambiguity, redundancy, erroneous join paths, and large table repositories mean there may be hundreds of result views. **Ver** uses novel techniques to reduce the number of views automatically (Section V), but there is a limit to automation. Semantic ambiguity requires involving users to obtain the final view. The VIEW-PRESENTATION component analyzes the views and generates questions that, when answered, help rank and select views. For example, this component will ask a user if they want "home address" or "work address" in their output when it detects both in the views. By asking questions, users learn more about the schemas and datasets available, and thus refine their preferences and discovery needs. A key challenge is that different users may be able to respond to different questions and that their preferences evolve as they interact with the system. **Ver**'s VIEW-PRESENTATION component is based on two design principles that cater to varied user needs.

• **Mixed-Initiative Interface Design:** Lack of knowledge about available datasets inhibits users from effectively querying the identified set of views. However, users can answer questions about their desired view. Each user has a different understanding of the requirements and finds different interfaces to be more appropriate depending on that. Therefore, users may need different interface designs to answer questions. For example, some users could recognize phone number from the area code, while others might look for a pattern across values. Because different users will be able to answer different questions, **Ver** supports different question interfaces, each asking a different question type. This is motivated by previous mixed-initiative designs [23], [24]. Besides, **Ver** learns which interface to offer to a user according to their previous actions.

• **Adapt to evolving users' knowledge:** Users' understanding of their discovery need evolves as they interact with questions and learn about the schemas and data contained in the repository. **Ver** is designed so users can change their

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:58:08 UTC from IEEE Xplore. Restrictions apply.